SATVIK PRASAD

satvikprasad@gatech.edu | satvikprasad.com | github.com/satvikprasad | US & AU Citizen

EDUCATION

Georgia Institute of Technology — Atlanta, GA

B.S. in CmpE + Mathematics, 4.0 GPA, John Martinson Honors Program — *Aug. 2025 – May 2029*

- Researching toroidal manifolds (`torformer`) for expressive, hierarchical self-attention in language modeling tasks under the mentorship of Emile Anand, affiliated with the ACO lab. Using `nanochat` as a harness.
- Incoming Team Lead, Big Data Big Impact, TerraTrends
- FinancialAI Project Team, AI@GT. Sponsored by Jane Street; researching RL approaches to slippage optimization problems.

Normanhurst Boys High School — Sydney, NSW, Australia

Valedictorian, 4.0 GPA (Converted from a 99.95 ATAR) — *Feb. 2019 – Oct. 2024*

EXPERIENCE

Cerebras Systems — May 2026 - Aug 2026

Codesign / Nextgen Performance Intern — *Sunnyvale, CA*

- Designed a new globally optimal inductive sizing algorithm for our `TreeAllReduce` primitive for the Kernel API, reducing latency by 24%, while maintaining throughput (used in production).
- Modelled the mapping of a new, low bandwidth, compute-bound binary tree `AllReduce` primitive used in `TopK` and `FlashAttention` kernels.
- Modeled a FlashAttention-inspired softmax variant from scratch, achieved a ~ 1.01 correlation ratio with respect to simulated RTL.
- Improved correlation ratios between cycle-accurate sims written in C, analytical models, and RTL simulators for critical kernels by an average of 189% (`GEMM`, `Softmax`, `TopK`, `FastPow2Exp`, etc.)
- Engineered new `python` tools to support an improved correlation workflow: a fabric visualizer, an `.fsdb` $\rightarrow$ E/C-Stage trace converter, and a high-performance batch performance validation tool.

Cognito Tuition — Feb 2025 - Aug 2025

Maths Ext. 1 & 2 Tutor and Educational Operations — *Sydney, Aus*

PROJECTS

tinf | *Tiny ONNX inference engine in Rust* — Dec 2025 - Current

- Implemented an ONNX parser using protobuf (`prost.rs`).
- Achieved end-to-end MNIST inference with a $\sim$98KB total memory footprint (0.46 ms / inference), and a 482KB binary size.
- Implemented `Conv2D`, `MaxPool`, `MatMul`, with appropriate kernel fusion, tiling, and strided access strategies for cache-friendly access.
- Designed zero-copy tensor views backed by a single bump allocator, avoiding per-op heap allocations entirely.
- Used buffer liveness analysis and graph coloring to reduce memory footprint for lightweight models by $\sim 1.5\times$.

Promptly | *Computer-use agent — Ergo Challenge @ HackGT25* — Sep 2025

- 10.5x lower latency comapred to Gemini 2.5 Computer Use Model (*state-of-art at the time*).
- Used `Omni-Parser` (pre-trained weights) and the RICO dataset to train a lightweight, local YOLO11n object detection model with 110x faster inference times than naive `OmniParser` on an M4.
- Wrote a hierarchical tree divide-and-conquer algorithm utilizing OpenCV for more accurate single-pass UI detection (50ms).
- Engineered a feedback-driven LLM event loop system for automatic task correction.

Peggiator | *Web-based, high performance & extensible music visualiser* — Jan 2025 – May 2025

- 4x speedup compared with the naive Chrome V8 runtime used a self-derived WASM implementation of the Fast-Fourier Transform (radix-4), `v128` SIMD & Web Workers for optimal cache access.
- Captured system audio using a helper-binary written in Swift and ScreenCaptureKit, injected audio data into a Vite + React sidebar component for the DOM.
- Used Zig and WebGL for 3D rendering of manipulated audio waveforms, wrote custom affine transformations and GLSL shaders to minimise memory bottlenecks.

AWARDS

- Winner, CalHacks 12.0 @ UC Berkeley (2000+ competitors); 99.95 ATAR (Top 50 in Australia); Honorable Mention, Intercollegiate Math Tournament @ Columbia
- State Rank 10th in NSW for Mathematics Ext. 2; Silver Medal AIO; UNSW Scientia Scholar; Best Maths Student in NSW Award

TECHNICAL SKILLS

Languages: C/C++ (STL), Python, Zig, Rust, OCaml, SQL, WebAssembly, TypeScript
Developer Tools: Git, Docker, `neovim`, Valgrind, `perf`
Libraries: CUDA, Metal, PyTorch, TensorFlow, OpenGL/WebGL, NumPy